

PostgreSQL: ASH con Captura Completa de I/O Kernel y Buffer Cache

Extensión del Esquema ASH: CPU, Lecturas/Escrituras Físicas (OS) y Bloques en Memoria

Para enriquecer el **ASH (Active Session History)** con métricas de Entrada/Salida (I/O), integramos tanto los bytes/bloques leídos y escritos a nivel del Sistema Operativo desde `pg_stat_kcache_detail` como las métricas de shared buffers/temp blocks desde `pg_stat_statements`.

PASO 1 Alteración de la Tabla de Historial (Campos de I/O)

Ejecuta el siguiente DDL para agregar las columnas de I/O físicas y de buffer en tu tabla de historial:

```
ALTER TABLE pg_stat_activity_history
ADD COLUMN IF NOT EXISTS io_reads_bytes bigint DEFAULT 0,
ADD COLUMN IF NOT EXISTS io_writes_bytes bigint DEFAULT 0,
ADD COLUMN IF NOT EXISTS shared_blks_hit bigint DEFAULT 0,
ADD COLUMN IF NOT EXISTS shared_blks_read bigint DEFAULT 0,
ADD COLUMN IF NOT EXISTS shared_blks_dirtied bigint DEFAULT 0,
ADD COLUMN IF NOT EXISTS temp_blks_read bigint DEFAULT 0,
ADD COLUMN IF NOT EXISTS temp_blks_written bigint DEFAULT 0;
```

PASO 2 Actualización de la Tarea en pg_cron (Captura Completa)

Actualizamos la función del job para almacenar métricas de CPU + I/O de disco del Kernel + Acceso a Bloques de PostgreSQL:

```
SELECT cron.schedule(
    'snapshot_pg_stat_history',
    '* * * * *',
    $$
    INSERT INTO pg_stat_activity_history (
        snapshot_time, pid, username, datname, state,
        wait_event_type, wait_event, cpu_user_seconds,
        cpu_system_seconds,
        io_reads_bytes, io_writes_bytes,
        shared_blks_hit, shared_blks_read, shared_blks_dirtied,
        temp_blks_read, temp_blks_written,
        query_id, query
    )
    SELECT
        now(),
        a.pid,
        a.username,
        a.datname,
        a.state,
        COALESCE(a.wait_event_type, 'CPU'),
        COALESCE(a.wait_event, 'CPU Executing'),
        ROUND(CAST(COALESCE(k.plan_user_time + k.exec_user_time, 0) AS numeric), 2),
        ROUND(CAST(COALESCE(k.plan_system_time + k.exec_system_time, 0) AS numeric), 2),
        -- I/O Física (Kernel via pg_stat_kcache)
        COALESCE(k.plan_reads + k.exec_reads, 0) AS io_reads_bytes,
        COALESCE(k.plan_writes + k.exec_writes, 0) AS io_writes_bytes,
        -- I/O Memoria & Temp (Buffer Cache via pg_stat_statements)
        COALESCE(s.shared_blks_hit, 0),
        COALESCE(s.shared_blks_read, 0),
        COALESCE(s.shared_blks_dirtied, 0),
        COALESCE(s.temp_blks_read, 0),
        COALESCE(s.temp_blks_written, 0),
        a.query_id,
        a.query
    FROM pg_stat_activity a
    LEFT JOIN pg_stat_statements s ON a.query_id = s.queryid
    LEFT JOIN pg_stat_kcache_detail k ON s.query = k.query AND a.datname = k.datname
    WHERE a.state = 'active'
    AND a.pid != pg_backend_pid();
```

```
$$  
);
```

Beneficio de la Captura Integrada de I/O:

- * **io_reads_bytes / io_writes_bytes:** Muestra el impacto REAL de I/O física que llegó al disco (Kernel).
- * **shared_blks_hit / read:** Permite calcular el Ratio de Eficiencia de Cache (`Hit Ratio`) por snapshot.
- * **temp_blks_*:** Detecta de inmediato si una consulta está desbordando a disco por falta de `work\_mem` (Sorts/HashJoins en temp).